

10PEARLS SHINE INTERNSHIP PROGRAM

DATA SCIENCE • COHORT 9

PEARLS AQI PREDICTOR

Production PM2.5 & AQI Forecasting Platform

Prepared by

Riyan Jamil

Data Science Intern

Internship Program	10Pearls Shine Internship Program
Track	Data Science
Cohort	Cohort 9
Project	Pearls AQI Predictor
Reference Location	Zafar Memon DHA, Karachi
Documentation	Final Project Report • August 2026

PROJECT LINKS

Live Project	https://ca-pearls-aqi-dashboard-prod.gentledesert-51a1063f.centralindia.azurecontainerapps.io/
GitHub Repository	https://github.com/riyanj220/pearls-aqi-predictor
LinkedIn	https://www.linkedin.com/in/riyan-jamil/
Email	riyanjamil220@gmail.com

End-to-end forecasting • MLOps • Azure cloud deployment

TABLE OF CONTENTS

Pearls AQI Predictor - Internship Documentation

No.	Section	Page
1	Executive Summary	1
2	Main Technology Stack	3
3	Why Was PM2.5 Forecast First?	4
4	Data Sources	5
5	Phase 1: Canonical Hourly Data Pipeline	6
6	Phase 2: Multi-Horizon Feature Engineering	7
7	Chronological Dataset Splits	9
8	Phase 3: Baselines, Model Training, and Selection	9
9	Models Evaluated During Initial Development	10
10	Why the Final Strategy Is Hybrid	12
11	Production Model Metrics	12
12	Phase 4: Explainability and Error Analysis	14
13	Phase 5: Live 72-Hour Inference	14
14	Phase 6: AQI Conversion and Alerts	15
15	Phase 7: FastAPI Service	16
16	Phase 8: Streamlit Dashboard	17
17	Phase 9: Feature Store, Registry, and Automated Retraining	19
18	Retraining and Champion/Challenger Logic	19
19	Model Registry	20
20	Why Hopsworks Was Migrated to Azure Blob	21
21	Azure Production Architecture	21
22	Azure Resource Organization	23
23	Scheduled Production Jobs	23
24	Immutable Forecast Publication	25
25	Serving Architecture	26
26	Monitoring and Operational Reliability	27
27	CI/CD and Containerization	27
28	Controlled Model Challenger Benchmark	28
29	Challenger Benchmark Results	29
30	Final Challenger Historical Test	30
31	Production Infrastructure Summary	31
32	Main Technical Problems Encountered and How They Were Resolved	32
33	End-to-End Production Flow	33
34	Conclusion	35

1. Executive Summary

During the internship, I developed and productionized an end-to-end PM2.5 and AQI forecasting system for the Zafar Memon DHA reference location in Karachi. I validated OpenAQ pollution data and Open-Meteo weather data, created a canonical hourly dataset, engineered 56 leakage-safe multi-horizon features, and built chronological train/validation/test datasets. I evaluated persistence baselines and multiple regression/boosting approaches and selected a hybrid strategy that uses current-value persistence for horizons 1–12 hours and XGBoost for horizons 13–72 hours.

I implemented explainability and error analysis, live 72-hour inference, PM2.5-to-AQI conversion, health guidance, and grouped alert episodes. I built a FastAPI service and professional Streamlit dashboard, then added MLOps functionality including a feature repository, model registry, versioning, automated retraining, champion/challenger evaluation, and controlled model promotion.

The complete system was containerized and deployed on Microsoft Azure using Azure Container Apps, Container Apps Jobs, Azure Blob Storage, Azure Container Registry, Managed Identity, and Azure Communication Services. Scheduled jobs update features hourly, publish a new forecast every six hours, evaluate retraining daily, and monitor production health hourly. Production artifacts and model versions are immutable and latest pointers are updated only after successful validation.

Finally, I performed a controlled model-family challenger benchmark covering historical controls, focused XGBoost, Histogram Gradient Boosting, Random Forest, and LightGBM. The strongest selected challenger improved validation performance but slightly worsened final historical test RMSE, so the existing hybrid production champion was retained. This benchmark confirmed that the production model remained the most appropriate choice rather than replacing it for a small validation-only gain.

The project began as a data-science forecasting problem and was progressively developed into a complete production system. It now includes:

Data engineering

- source discovery;
- API ingestion;
- historical collection;
- schema validation;
- time alignment;
- quality rules;
- canonical datasets;
- Incremental updates.

Data science

- exploratory analysis;

- feature engineering;
- leakage prevention;
- chronological evaluation;
- persistence baselines;
- model comparison;
- controlled tuning;
- error analysis;
- explainability;
- challenger benchmarking.

Machine learning

- multi-horizon PM2.5 regression;
- Ridge;
- HistGradientBoosting;
- XGBoost;
- Random Forest challenger evaluation;
- LightGBM challenger evaluation;
- hybrid routing strategy;
- model artifact serialization;
- model metadata.

MLOps

- feature repository;
- model registry;
- model versioning;
- production pointers;
- retraining eligibility;
- champion/challenger policy;
- controlled promotion;
- immutable artifacts;
- model lineage.

Backend engineering

- FastAPI;
- health/readiness endpoints;
- artifact-backed serving;
- public-safe error handling.

Frontend/data application

- professional Streamlit dashboard;
- forecast visualization;
- AQI analysis;
- alert interpretation;
- technical system-status presentation.

DevOps/cloud

- Docker;
- Azure Container Registry;
- Azure Container Apps;
- Azure Container Apps Jobs;
- Blob Storage;
- Managed Identity;
- scheduled workloads;
- GitHub Actions;
- production validation;
- monitoring;
- failure notifications;
- cost optimization.

2. Main Technology Stack

Data Science

- Python
- Pandas
- NumPy
- PyArrow

- scikit-learn
- XGBoost
- LightGBM for controlled challenger evaluation
- Joblib
- SHAP / model explainability tooling
- Matplotlib
- Plotly

Application

- FastAPI
- Uvicorn
- Pydantic/Pydantic Settings
- Streamlit

MLOps / infrastructure

- Docker
- Docker Compose
- Azure Container Registry
- Azure Container Apps
- Azure Container Apps Jobs
- Azure Blob Storage
- Azure Managed Identity
- Azure Communication Services
- GitHub Actions
- Hopsworx as optional/demo backend
- uv for Python dependency/environment management

3. Why Was PM2.5 Forecast First?

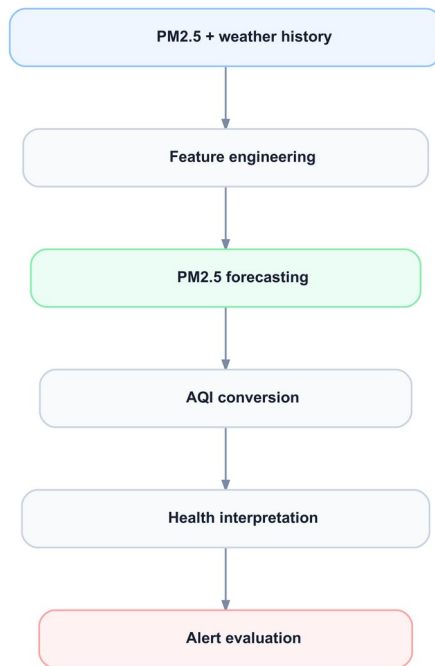
The machine-learning model predicts PM2.5 concentration in $\mu\text{g}/\text{m}^3$, rather than predicting AQI directly.

This separation was chosen because:

- PM2.5 is the actual measured pollutant signal;
- PM2.5 is continuous and therefore suitable for regression;

- the AQI layer can remain deterministic and transparent;
- AQI standard changes can be handled separately from model retraining;
- model evaluation can be performed directly in pollutant units;
- one model output can support several interpretation layers.

The overall logic is therefore:



This architecture keeps the statistical forecasting problem separate from the public-health interpretation layer.

4. Data Sources

4.1 PM2.5 — OpenAQ

OpenAQ was used as the pollution-data source.

During the discovery phase, multiple candidate sensors and locations were investigated rather than blindly choosing the first available source. The final reference sensor was selected after validating:

- temporal availability;
- data completeness;
- hourly coverage;
- duplicate behavior;
- missing periods;
- implausible readings;

- zero-reading patterns;
- suitability for the required historical range.

The project ultimately used the validated Zafar Memon DHA reference location.

4.2 Weather — Open-Meteo

Open-Meteo was used for:

- historical weather;
- forecast weather for live inference.

Weather variables included:

- temperature;
- relative humidity;
- dew point;
- surface pressure;
- precipitation;
- rain;
- cloud cover;
- visibility;
- wind speed;
- wind direction;
- wind gusts.

Historical weather was used for model development, while live inference reconstructs the same target-hour weather feature contract from Open-Meteo forecast data.

4.3 Time Standard

All internal timestamps were normalized to timezone-aware UTC.

This was important because the project joins pollution observations, historical weather, forecast weather, training targets, live forecasts, scheduled jobs, and cloud artifacts. A single internal timezone prevents ambiguous joins and daylight/timezone errors.

Display timezones are handled only in the presentation layer.

5. Phase 1: Canonical Hourly Data Pipeline

The first major milestone was producing a reliable canonical hourly dataset.

Completed work included:

- validating the selected OpenAQ PM2.5 source;

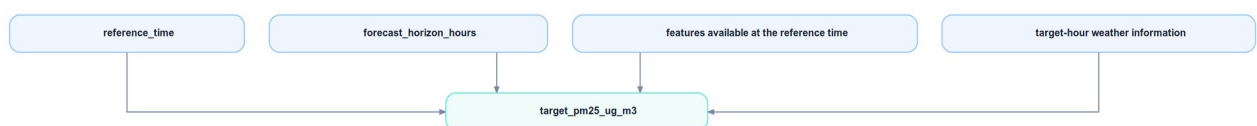
- collecting historical PM2.5 observations;
- collecting Open-Meteo historical weather;
- validating at least 72 hours of forecast-weather availability;
- normalizing timestamps to hourly UTC;
- removing duplicate records;
- checking chronological ordering;
- identifying missing hours;
- evaluating long missing runs;
- applying pollution quality rules;
- treating exact PM2.5 zero values as missing where appropriate;
- retaining plausible high pollution readings rather than automatically removing extremes;
- aligning PM2.5 and weather on hourly timestamps;
- Saving the validated canonical dataset.

The goal was not only to collect data but to create a trustworthy input contract for every later phase. A model can only be credible when the historical dataset is traceable, chronological, and validated.

6. Phase 2: Multi-Horizon Feature Engineering

The forecasting requirement was not a single next-hour prediction. The system needed 72 direct forecast horizons.

Each modeling row therefore represents:



6.1 Feature Families

The feature engineering pipeline produced 56 ordered model features.

The main feature families included:

PM2.5 history

- current PM2.5;
- PM2.5 lag features;
- recent historical values.

Rolling pollution behavior

- rolling averages;
- rolling variability;
- trend-related features;
- recent change features.

Current weather

Weather measured at the reference timestamp.

Target-hour weather

Weather conditions associated with the future hour being predicted.

This is particularly important for multi-horizon prediction because horizon 60 should not necessarily use the same weather assumptions as horizon 2.

Temporal features

- hour of day;
- day of week;
- other calendar information;
- cyclical time encoding.

Wind encoding

Wind direction was transformed into model-friendly components instead of treating degrees as a simple linear number.

Forecast horizon

forecast_horizon_hours itself is an input feature so the learned model can distinguish short-, medium-, and long-range behavior.

6.2 Leakage Prevention

A strict feature contract was created and validated.

Checks included:

- target column exists;
- all required features exist;
- identifier columns are not model inputs;
- target column is not a model input;
- future PM2.5-derived features are excluded;
- feature order is deterministic;

- no unexpected missing feature values;
- no duplicate reference-time/horizon keys;
- horizon coverage is 1–72;
- split chronology is preserved.

This was essential because multi-horizon forecasting can easily introduce target leakage if future PM2.5 information is accidentally exposed to the model.

7. Chronological Dataset Splits

Random train/test shuffling was intentionally avoided because this is a time-dependent forecasting problem.

The finalized development split was chronological.

Training Set

- **Reference start:** 9 July 2025
- **Reference end:** 18 March 2026
- **Rows:** 364,798

Validation Set

- **Reference start:** 21 March 2026
- **Reference end:** 27 May 2026
- **Rows:** 71,256

Test Set

- **Reference start:** 30 May 2026
- **Reference end:** 23 July 2026
- **Rows:** 76,813

This approach better approximates the real deployment situation:

learn from the past, select using a later period, and finally evaluate on an even later unseen period.

8. Phase 3: baselines, Model Training, And Selection

8.1 Why Persistence Baselines Were Necessary

A forecasting model should not be considered useful simply because it has a reasonable RMSE.

For air pollution, recent values can be highly persistent, especially over short forecast horizons.

Two simple baselines were therefore created.

Current-value persistence

future PM2.5 = PM2.5 observed at the reference timestamp

Previous-day persistence

future PM2.5 = PM2.5 observed 24 hours earlier

These baselines created a realistic minimum standard that machine-learning models had to beat.

8.2 Metrics

The primary metrics were:

MAE (Mean Absolute Error)

It answers: On average, how far is the prediction from the actual PM2.5 value?

RMSE (Root Mean Squared Error)

It penalizes larger errors more strongly and was used as the main model-selection metric.

R² (Coefficient of determination)

It provides information about explained variation, but it was not used in isolation because low-variance chronological periods can produce weak R² even when MAE/RMSE improve relative to operational baselines.

8.3 Horizon-based Evaluation

Performance was evaluated not only overall but also across:

- 1–6 hours;
- 7–12 hours;
- 13–24 hours;
- 25–48 hours;
- 49–72 hours.

This revealed an important behavior:

persistence was very competitive at immediate horizons, while learned nonlinear models provided more value as the forecast horizon increased.

That observation directly led to the final hybrid strategy.

9. Models Evaluated During Initial Development

The initial model-development work included:

- persistence baselines;
- Ridge Regression;

- Histogram Gradient Boosting;
- XGBoost;
- controlled XGBoost refinements.

9.1 Ridge Regression

Ridge Regression was used as a simple regularized linear model.

Its purpose was not only to compete for production but to answer whether the engineered feature set could be modeled sufficiently with a linear relationship.

It used:

- StandardScaler;
- Ridge regularization;
- a scikit-learn pipeline.

The result demonstrated that the PM2.5 forecasting problem contained meaningful nonlinear relationships that a simple linear model did not capture sufficiently.

9.2 Histogram Gradient Boosting

Histogram Gradient Boosting was then evaluated as a nonlinear tree-based method.

It improved the ability to model:

- PM2.5 history;
- rolling pollution patterns;
- weather interactions;
- temporal effects;
- forecast-distance behavior.

9.3 XGBoost

XGBoost was evaluated with chronological validation and early stopping.

A controlled set of configurations was tested rather than running an unbounded hyperparameter search.

The best long-range configuration became: `xgboost_shallower`

Important properties included:

- shallower trees;
- controlled learning rate;
- subsampling;
- feature subsampling;

- L1/L2 regularization;
- chronological validation;
- early stopping.

The selected saved model reached its best boosting iteration at: 323

10. Why The Final Strategy Is Hybrid

The key modeling result was that no single method was clearly best across all 72 horizons.

Short horizons behaved differently from medium/long horizons.

The final production strategy therefore became:



Internal strategy name:

hybrid_persistence_1_12_xgboost_shallower_13_72

This strategy was selected because:

1. current-value persistence performed best for immediate horizons;
2. XGBoost provided greater value at medium and long horizons;
3. the hybrid achieved the lowest overall validation RMSE among the evaluated initial strategies;
4. the routing rule is deterministic and easy to reproduce;
5. XGBoost produced no negative test predictions in the selected evaluation;
6. the architecture remained simple enough for reliable production inference.

This is an important project decision.

The system did not use XGBoost everywhere simply because it was the most sophisticated model. It used the best validated approach for each region of the forecast horizon.

11. Production Model Metrics

Current production model family: XGBRegressor

Long-horizon configuration: xgboost_shallower

Production model registry version: Version 1

Input feature count: 56

Best boosting iteration: 323

11.1 Validation Performance of the Production Hybrid

- **MAE:** 6.6956 $\mu\text{g}/\text{m}^3$
- **RMSE:** 9.4196 $\mu\text{g}/\text{m}^3$
- **R²:** 0.0666

11.2 Final Historical Test Performance

- **MAE:** 3.7155 $\mu\text{g}/\text{m}^3$
- **RMSE:** 4.9756 $\mu\text{g}/\text{m}^3$
- **R²:** -0.0040

11.3 Test Baselines

Current persistence

- **MAE:** 3.8354
- **RMSE:** 5.2831
- **R²:** -0.1319

Previous-day persistence

- **MAE:** 4.5438
- **RMSE:** 6.1175
- **R²:** -0.5177

The production hybrid achieved approximately:

5.82% lower test RMSE than current value persistence.

11.4 Interpreting the Low-test R²

The test R² was approximately zero/slightly negative.

This was documented rather than hidden.

The chronological test period had relatively limited PM2.5 variance. In such a period, R² can look weak even when absolute forecast error improves.

Therefore, the project does not describe the model using a misleading generic “accuracy percentage.”

Instead, performance is presented through:

- MAE;
- RMSE;
- improvement over persistence;

- horizon-based stability;
- severe-condition analysis;
- prediction validity.

This is a more appropriate evaluation framework for the forecasting problem.

12. Phase 4: Explainability And Error Analysis

The selected model was not moved directly into production after model selection.

A dedicated explainability and error-analysis stage was completed.

It included:

- reproduction of final test predictions;
- global model explanations;
- local prediction explanations;
- feature-importance analysis;
- residual analysis;
- error by forecast horizon;
- error by pollution level;
- largest-error investigation;
- prediction-bias investigation;
- distribution comparisons;
- documentation of limitations;
- readiness assessment for live inference.

This stage was important because a production forecasting model should be understood not only by average error metrics but also by how and where it fails.

A known behavior was that error generally increases with forecast distance.

Another documented limitation is that tree-based ensemble models may underrepresent extreme PM2.5 events because predictions tend to regress toward ranges represented strongly in the training data.

13. Phase 5: Live 72-hour Inference

The next objective was to reproduce the training feature contract using live data.

The live inference pipeline:

1. retrieves recent PM2.5 history;
2. retrieves current and forecast weather;

3. selects a valid prediction reference timestamp;
4. reconstructs all 56 ordered production features;
5. verifies training-serving feature parity;
6. generates exactly 72 forecast rows;
7. routes horizons 1–12 to persistence;
8. routes horizons 13–72 to the registered XGBoost estimator;
9. validates predictions;
10. saves inference metadata and validation reports.

The live pipeline therefore reproduces the same feature definitions used during model development rather than creating a separate ad-hoc production feature path.

14. Phase 6: AQI Conversion and Alerts

After PM2.5 prediction, the system produces a public-health interpretation layer.

The AQI pipeline generates:

- indicative hourly PM2.5-based AQI;
- rolling 24-hour PM2.5-based AQI where a complete window exists;
- AQI categories;
- health messages;
- recommended actions;
- operational alert levels;
- alert trigger basis;
- hazardous flags;
- grouped alert episodes;
- forecast summaries.

AQI categories used by the dashboard include:

- Good;
- Moderate;
- Unhealthy for Sensitive Groups;
- Unhealthy;
- Very Unhealthy;
- Hazardous.

An important distinction is maintained between:

Indicative hourly AQI

An AQI interpretation derived from one forecast hour's predicted PM2.5 concentration.

Rolling 24-hour AQI

An interpretation based on a trailing 24-hour PM2.5 window, using observed and forecast values where a complete window is available.

Consecutive alert hours are grouped into episodes containing information such as:

- start time;
- end time;
- duration;
- maximum AQI;
- maximum AQI category;
- maximum alert level;
- health guidance;
- sensitive-group impact;
- general-population impact;
- hazardous condition flag.

15. Phase 7: FastAPI Service

FastAPI was added as the serving layer.

A key architecture decision was:

API requests do not run expensive model inference or ingestion.

The scheduled pipelines produce validated artifacts first. FastAPI reads the latest successfully published artifact.

Important production routes include:

```
/api/v1/health/live  
/api/v1/health/ready  
/api/v1/forecast
```

The API design includes:

- health checks;
- readiness checks;
- forecast freshness;

- contract validation;
- public-safe errors;
- no exposure of internal local paths or secrets.

This separation improves reliability and response time.

If a data source fails temporarily, the API can continue serving the previous validated forecast, subject to freshness/readiness reporting, rather than generating broken output during a user request.

16. Phase 8: Streamlit Dashboard

A Streamlit dashboard was developed on top of the FastAPI API.

The final dashboard contains three major user-facing areas.

16.1 Forecast

The Forecast page shows:

- current forecast AQI;
- AQI category;
- health message;
- forecast freshness;
- 72-hour horizon;
- PM2.5 summary metrics;
- worst forecast AQI;
- alert-hour count;
- PM2.5 trajectory;
- AQI category timeline;
- indicative hourly AQI;
- rolling 24-hour AQI;
- selected-hour inspection;
- hourly forecast data;
- forecast methodology;
- location context.

The UI was later redesigned into a professional environmental-monitoring console.

Normal “no-alert” conditions are not displayed as large green alert banners because zero alerts is a normal state, not an exceptional success event.

16.2 Alerts

The Alerts page was redesigned to provide value even when there are zero active alerts.

It contains:

- current alert posture;
- active alert count;
- upcoming alerts;
- alert episodes;
- hazardous episodes;
- AQI category guide;
- hourly versus rolling AQI explanation;
- alert methodology;
- episode cards;
- health guidance.

This makes the alert system understandable and demonstrates that alert generation is based on the forecast pipeline rather than hard-coded UI behavior.

16.3 System Status

The System Status page provides technical transparency around:

- API health;
- forecast freshness;
- artifact consistency;
- current forecasting strategy;
- model metrics;
- model baselines;
- chronological dataset split;
- production pipeline design;
- data sources;
- infrastructure;
- monitoring location map.

A later improvement identified for this page is to source all production-model and Azure-job execution details dynamically rather than keeping a local UI snapshot for some model/pipeline fields.

17. Phase 9: feature Store, Registry, And Automated Retraining

The project then moved beyond static local model files.

Phase 9 introduced MLOps abstractions for:

- feature storage;
- model registration;
- historical backfill;
- incremental feature updates;
- training-dataset versioning;
- automated retraining eligibility;
- candidate training;
- champion/challenger evaluation;
- controlled production-model resolution;
- model lineage;
- local fallback behavior.

The initial implementation used **Hopsworks Feature Store and Model Registry**.

This was useful for demonstrating:

- feature-group concepts;
- versioned feature views;
- model versions;
- production-stage resolution;
- feature lineage;
- retraining workflows.

The production architecture was later migrated away from Hopsworks to Azure Blob Storage.

18. Retraining And Champion/Challenger Logic

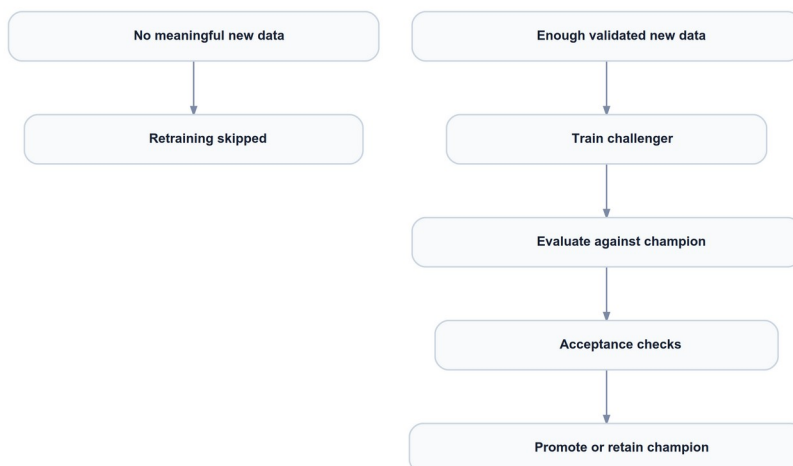
The system includes a retraining workflow designed to run automatically while avoiding unnecessary model replacement.

Retraining logic can evaluate:

- whether new valid observations exist;
- whether sufficient new target rows are available;

- feature completeness;
- data quality;
- current model age;
- forced-retraining configuration.

Possible behavior includes:



A scheduled retraining job succeeding does not necessarily mean a new model is promoted.

The model promotion process is intentionally conservative.

A candidate should only become production when it passes validation, artifact, prediction, horizon, and model-quality checks.

19. Model Registry

The model registry stores immutable model versions.

The current production model is:

```

pearls_aqi_pm25_forecaster
Version 1

```

The registry includes artifacts such as:

```

best_model.joblib
model_feature_columns.json
model_metadata.json
model_selection_report.json
registry_metadata.json
manifest.json

```

A production pointer identifies which immutable version should be loaded.

Conceptually:

```

model-registry/
├── pearls_aqi_pm25_forecaster/
│   ├── production/
│   │   └── pointer.json
│   └── versions/
│       ├── 1/
│       │   ├── best_model.joblib
│       │   ├── model_metadata.json
│       │   └── model_feature_columns.json
│       └── ...

```

This means applications do not assume that “version 1” will always be production.

They resolve the production pointer.

That design supports future champion promotion and rollback.

20. Why Hopsworks Was Migrated to Azure Blob

The project originally integrated Hopsworks as both:

- feature store;
- model registry.

During production hardening, the architecture was migrated so that production no longer depends on Hopsworks.

Production now uses:

```

FEATURE_STORE_BACKEND=azure_blob
MODEL_REGISTRY_BACKEND=azure_blob
MODEL_LOADING_MODE=AZURE_BLOB_REGISTRY
ARTIFACT_BACKEND=azure_blob

```

Hopsworks remains an optional/demo backend rather than a production dependency.

Reasons for the migration

- After the 3 days of initial project completion and jobs running, my free tier credits got finished and I was told to wait for the next month.
- Even when my free tier was not full, the jobs were taking more than 30 minutes which normally should have been completed under 5 minutes maximum causing timeout issues and disrupting the data ingestion.

This was not a redesign of the forecasting model.

The backend abstraction was preserved, so production storage changed while core data-science behavior remained stable.

21. Azure Production Architecture

The final production architecture uses Microsoft Azure.

21.1 Main Azure Components

Azure Container Registry

Stores versioned Docker images for:

- FastAPI;
- Streamlit dashboard;
- pipeline jobs.

Registry:

```
walpole.azurecr.io
```

Azure Container Apps

Used for continuously available/read-serving applications:

- FastAPI;
- Streamlit dashboard.

Production applications:

```
ca-pearls-aqi-api-prod  
ca-pearls-aqi-dashboard-prod
```

Azure Container Apps Jobs

Used for scheduled batch workloads:

```
job-pearls-aqi-features-prod  
job-pearls-aqi-forecast-prod  
job-pearls-aqi-retraining-prod  
job-pearls-aqi-monitoring-prod
```

Azure Blob Storage

Storage account:

```
stpearlsaqiriyan
```

Production container:

```
artifacts-prod
```

Blob Storage contains:

- production feature datasets;
- model registry;
- AQI forecast artifacts;
- latest pointers;
- production-health snapshots;

- monitoring outbox/receipts;
- validation metadata.

Managed Identity

A user-assigned managed identity is used so production workloads can access Azure resources without embedding storage passwords in application code.

Production identity:

```
id-pearls-aqi-prod
```

Azure Communication Services

Used for production monitoring notifications through email.

22. Azure Resource Organization

Production resource group:

```
rg-pearls-aqi-prod
```

A shared Container Apps Environment is used:

```
cae-pearls-aqi-staging
```

Although its name contains “staging,” it became the shared execution environment because the Azure for Students subscription had practical resource/quota constraints.

This is an example of adapting the architecture to real cloud constraints without compromising workload isolation at the application/job level.

23. Scheduled Production Jobs

Four production jobs automate the end-to-end operational lifecycle.

All schedules are interpreted in UTC.

23.1 Feature Synchronization Job

Resource:

```
job-pearls-aqi-features-prod
```

Schedule:

```
15 * * * *
```

Frequency: Hourly, at minute 15.

Purpose:

- refresh PM2.5 observations;
- refresh weather observations;

- update production feature datasets;
- maintain the Azure Blob feature repository.

Production feature-store structure includes datasets such as:

```
feature-store/
├─ pm25_hourly_observations/
├─ weather_hourly_observations/
└─ pm25_hourly_features/
```

Each dataset is represented by durable data and metadata artifacts.

23.2 Forecast Publication Job

Resource:

```
job-pearls-aqi-forecast-prod
```

Schedule:

```
0 */6 * * *
```

Frequency: Every six hours.

Purpose:

1. resolve the production model;
2. load validated features;
3. retrieve required live inputs;
4. generate 72 PM2.5 predictions;
5. convert predictions to AQI;
6. evaluate alerts;
7. validate the full artifact package;
8. publish an immutable forecast run;
9. advance the latest pointer only after success.

23.3 Retraining job

Resource:

```
job-pearls-aqi-retraining-prod
```

Schedule:

```
30 3 * * *
```

Frequency: Daily at 03:30 UTC.

Purpose:

- evaluate retraining eligibility;
- build/validate a candidate when required;
- compare challenger and champion;
- retain or promote according to acceptance policy;
- update registry metadata only when promotion is justified.

Daily execution does not mean forced daily production model changes.

23.4 Monitoring job

Resource:

```
job-pearls-aqi-monitoring-prod
```

Schedule:

```
45 * * * *
```

Frequency: Hourly, at minute 45.

Purpose:

- inspect production health;
- create durable health snapshots;
- track incident state;
- deduplicate repeated incidents;
- manage notification outbox;
- record delivery receipts;
- Send Azure Communication Services email when required.

24. Immutable Forecast Publication

Forecast artifacts are treated as immutable runs.

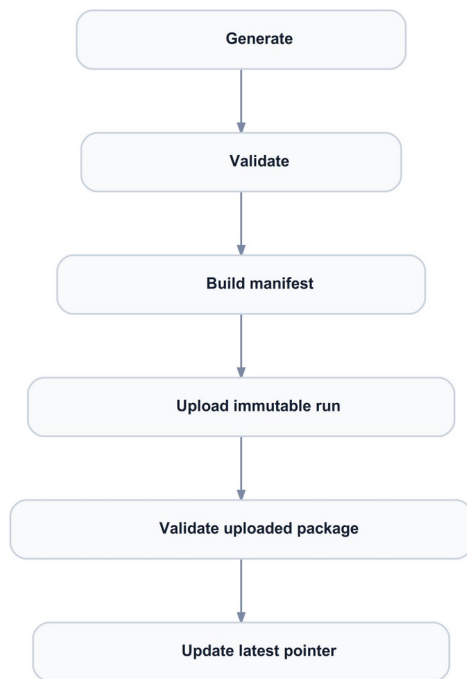
A typical production run contains:

```
aqi/runs/<run-id>/
├─ alert_episodes.json
├─ aqi_forecast_summary.json
├─ aqi_metadata.json
├─ live_pm25_aqi_forecast.parquet
├─ manifest.json
└─ validation report
```

The latest successful forecast is identified through:

```
aqi/latest/pointer.json
```

Publication follows:



The latest pointer is never advanced before the new run is fully validated.

This provides an important reliability property: A failed forecast run cannot replace the previous known-good forecast.

25. Serving Architecture

The production runtime separates batch computation from user traffic.



This separation was deliberate.

Benefits

- no expensive model inference inside HTTP requests;
- predictable API response time;
- failed source collection does not immediately destroy the user experience;
- artifacts can be validated before becoming public;
- API and dashboard can scale independently from pipelines;
- batch jobs can use different resource policies;
- individual jobs can be retried or investigated without redeploying serving applications.

26. Monitoring And Operational Reliability

Production monitoring was implemented as more than a simple log statement.

Health state is stored in Azure Blob.

Example structure:

```
production-health/  
├─ runs/<run-id>/  
├─ latest/  
│   └─ pointer.json  
└─ notifications/  
    ├── outbox.json  
    └─ receipts/
```

The monitoring design includes:

- health snapshots;
- incident state;
- incident deduplication;
- durable notification outbox;
- delivery receipt tracking;
- Azure Communication Services email integration.

This reduces the risk of repeated notifications for the same unresolved condition and provides traceable operational history.

27. CI/CD and Containerization

The system was containerized with separate images for:

- FastAPI;
- Streamlit;
- pipeline workloads.

Repository-level deployment artifacts include:

```
Dockerfile.api  
Dockerfile.dashboard  
Dockerfile.pipeline
```

Docker Compose configurations support local/production-style execution.

GitHub Actions and deployment scripts were introduced for:

- automated tests;
- image building;

- image publication;
- deployment validation;
- immutable release tracking.

Images are tagged with immutable source/version identifiers rather than relying only on a mutable latest tag.

This supports reproducibility and rollback.

28. Controlled Model Challenger Benchmark

After the full production system was complete, a final concern remained:

Had enough model families been challenged before permanently accepting the production XGBoost strategy?

Instead of rewriting Phase 3 or modifying production, a separate controlled benchmark was created:

`notebooks/12_model_challenger_benchmark.ipynb`

The benchmark was deliberately isolated from production.

28.1 Benchmark Phases

The benchmark included:

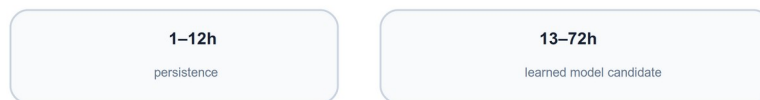
- experiment-contract and safety validation;
- exact production champion reproduction;
- standardized evaluation utilities;
- historical control reproduction;
- persistence controls;
- Ridge control;
- Histogram Gradient Boosting control;
- saved XGBoost control;
- historical models under the hybrid strategy;
- focused 13–72h XGBoost challenger;
- focused HistGradientBoosting challengers;
- Random Forest challenger;
- stage-1 validation leaderboard;
- LightGBM external challenger;
- robustness analysis;

- validation selection freeze;
- one-time final historical test;
- final production-impact decision;
- saved benchmark artifacts.

28.2 Why The Benchmark Preserved The Hybrid Architecture

The experiment did not assume that a new model had to replace persistence for all 72 hours.

The existing architecture was treated as:



This was important because the original evidence already showed that persistence was difficult to beat at immediate horizons.

The challengers therefore competed primarily for the learned 13–72h region while the full hybrid metrics were still evaluated.

29. Challenger Benchmark Results

The final validation leaderboard included the production champion and strongest challengers.

Production Champion

Family: XGBRegressor

Strategy: persistence 1–12h + XGBoost 13–72h

Validation:

- **MAE:** 6.6956
- **RMSE:** 9.4196
- **R²:** 0.0666

LightGBM finalist

Candidate: lightgbm_l1

Validation:

- **MAE:** 6.7167
- **RMSE:** 9.3038
- **R²:** 0.0894
- **RMSE improvement vs champion:** 1.23%

- **MAE improvement vs champion:** -0.31%
- **Training time:** approximately 80.8 seconds
- **Prediction time:** approximately 0.93 seconds
- **Model size:** approximately 1.13 MiB
- **Negative raw predictions:** 0
- **Clipped predictions:** 0

LightGBM achieved the best validation RMSE among the final leaderboard entries but slightly worsened MAE and was not the frozen challenger selected for the one-time test.

Selected challenger

Candidate: `focused_histgb_h2_absolute`

Family: `HistGradientBoostingRegressor`

Training scope: 13–72h

Validation:

- **MAE:** 6.5846
- **RMSE:** 9.3066
- **R²:** 0.0889
- **RMSE improvement vs production:** 1.20%
- **MAE improvement vs production:** 1.66%

It was selected because it provided a balanced validation improvement rather than optimizing only one metric.

30. Final Challenger Historical Test

The selected `HistGradientBoosting` challenger received the one-time historical test comparison.

Production Champion

- **MAE:** 3.7155
- **RMSE:** 4.9756
- **R²:** -0.0040

Selected Histgradient boosting Challenger

- **MAE:** 3.6284
- **RMSE:** 4.9873
- **R²:** -0.0087

Compared with production:

- **MAE improved by approximately 2.34%**
- **RMSE worsened by approximately 0.24%**

There were **0 severe-PM2.5 samples** in that historical test period, so the benchmark could not use the test set to claim improved extreme-event performance.

Because RMSE remained the primary forecast-error criterion and the challenger did not beat the production champion on final test RMSE, it was not promoted.

Final benchmark decision:

```
KEEP_PRODUCTION_CHAMPION
```

The benchmark also recorded:

- no additional production training required;
- no production pointer change required;
- no registry update required;
- no production architecture change required.

This result increased confidence in the existing production model because it was retained after a controlled, broader model-family challenge rather than simply left unchanged by default.

31. Production Infrastructure Summary

Cloud

Microsoft Azure

Container Registry

Azure Container Registry:

```
walpole.azurecr.io
```

Read-serving Workloads

```
ca-pearls-aqi-api-prod
ca-pearls-aqi-dashboard-prod
```

Scheduled Workloads

```
job-pearls-aqi-features-prod
job-pearls-aqi-forecast-prod
job-pearls-aqi-retraining-prod
job-pearls-aqi-monitoring-prod
```

Durable Storage

```
stpearlsaqiriyan
```

Identity

id-pearls-aqi-prod

Notifications

Azure Communication Services Email

32. Main Technical Problems Encountered and How They Were Resolved

Problem 1: selecting A Trustworthy PM2.5 Source

Risk

Choosing an incomplete or unreliable sensor would invalidate all later work.

Resolution

Multiple sensor candidates were evaluated for historical coverage, missing periods, quality behavior, and suitability before selecting the production reference source.

Problem 2: Missing and suspicious PM2.5 values

Risk

Zero readings and missing hours can represent source/data issues rather than real pollution.

Resolution

Data-quality rules were implemented. Exact PM2.5 zero values were treated as missing in the validated historical workflow, while plausible high pollution values were preserved.

Problem 3: Multi-horizon leakage risk

Risk

Direct 72-hour modeling can accidentally include information from the future.

Resolution

A deterministic feature contract was created, and explicit leakage checks ensured that no future PM2.5-derived values entered model inputs.

Problem 4: One model was not best at every horizon

Risk

Using XGBoost everywhere would unnecessarily lose the strength of persistence at short horizons.

Resolution

A hybrid horizon-aware strategy was created:

Problem 5: Reproducing training features in live inference

Risk

Training-serving skew can invalidate deployed forecasts.

Resolution

The production pipeline loads the saved ordered feature contract and validates the exact 56-column live feature matrix before prediction.

Problem 6: Avoiding invalid publication

Risk

A partially generated forecast could be exposed if storage is updated too early.

Resolution

Immutable run directories and atomic latest-pointer advancement were implemented.

Problem 7: Scheduled retraining should not blindly replace models

Risk

Frequent automated retraining can make production worse.

Resolution

Champion/challenger acceptance logic was used. A successful retraining job can still retain the existing champion.

Problem 8: Concern that XGBoost had not been challenged broadly enough

Risk

The original champion may have been accepted without enough model-family breadth.

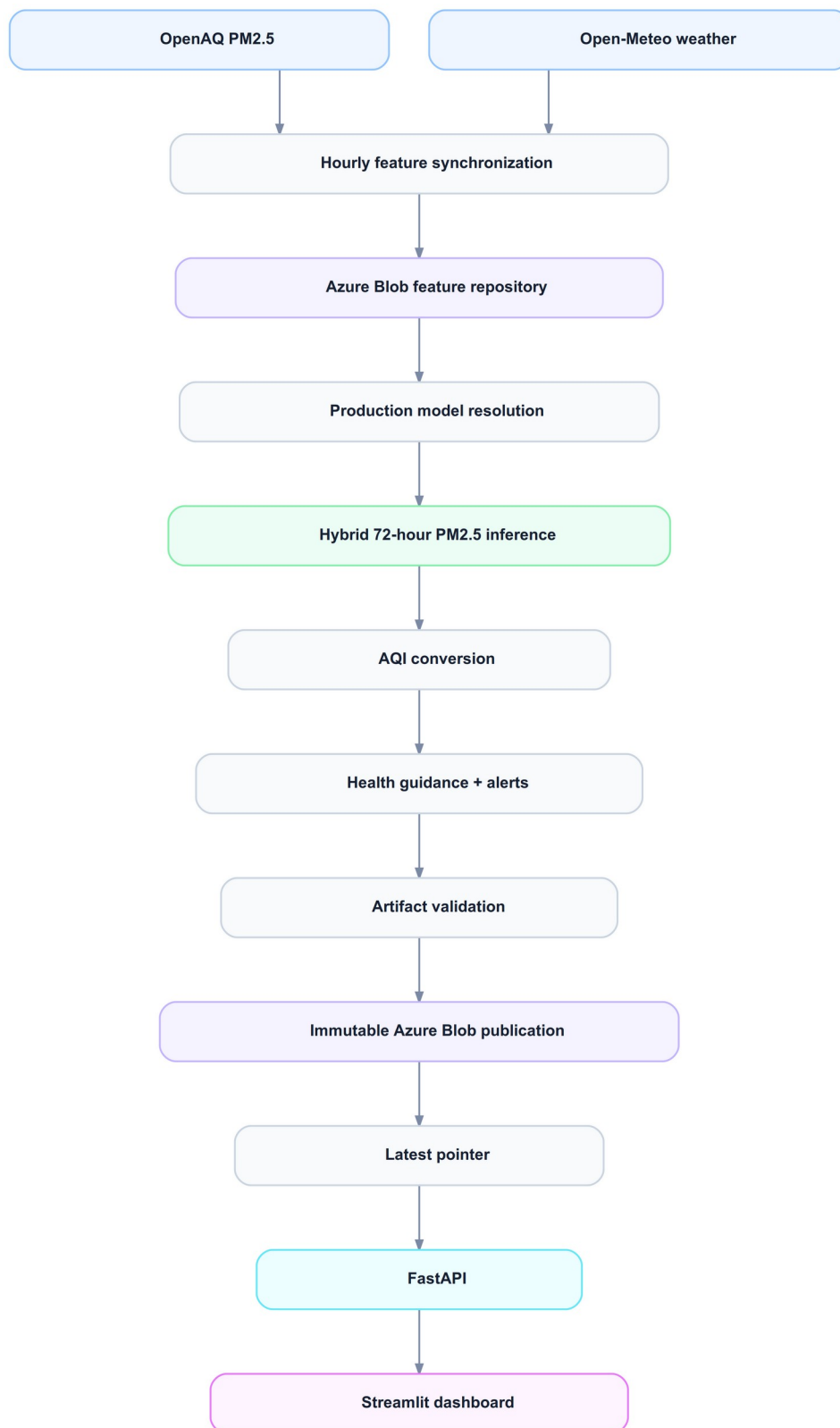
Resolution

A controlled post-production benchmark evaluated historical controls, focused XGBoost, HistGradientBoosting, Random Forest, and LightGBM while protecting the test-selection policy.

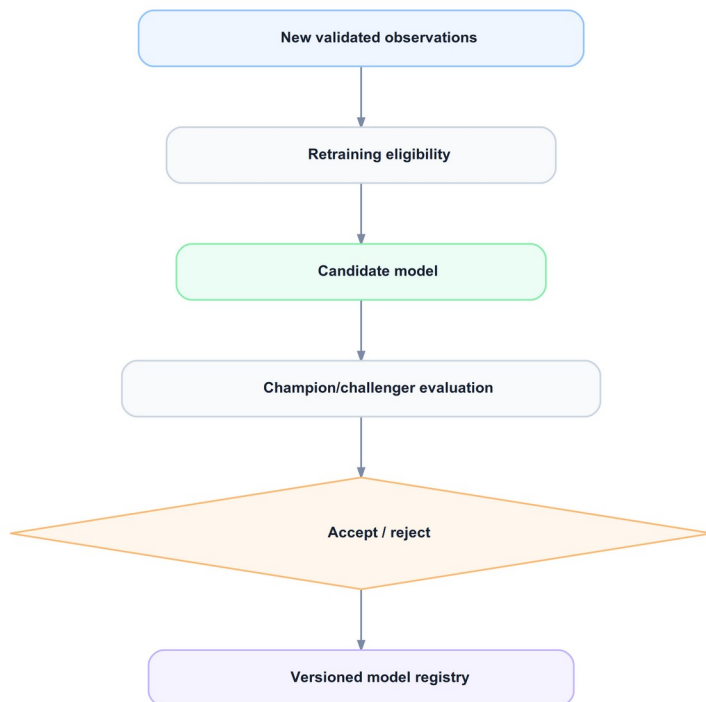
Result: The production champion remained best and was retained.

33. End-to-End Production Flow

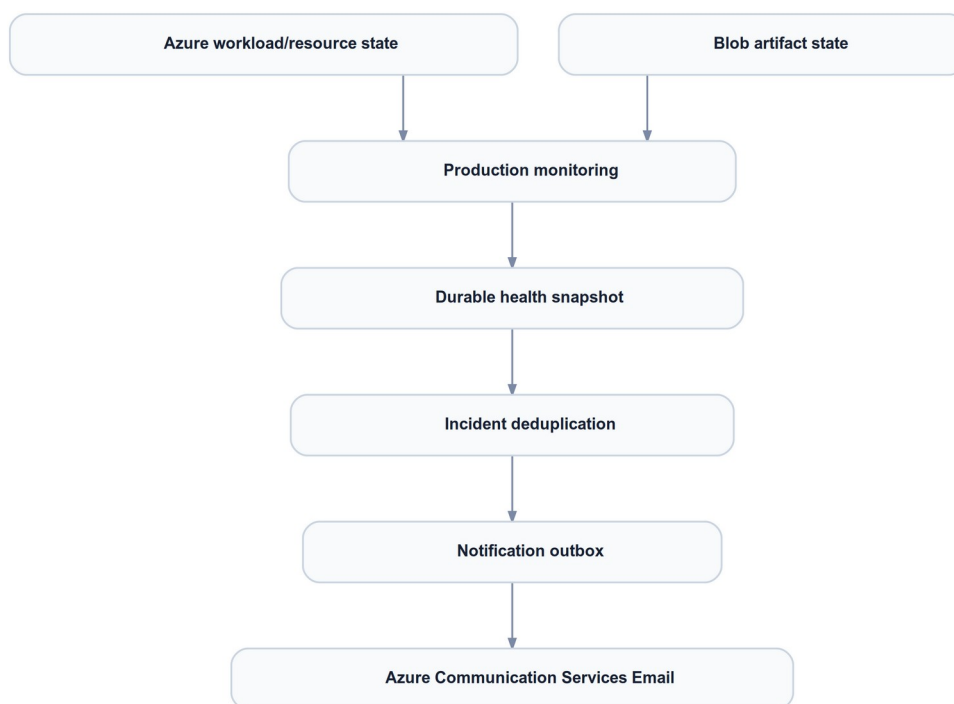
The final operational flow is:



Parallel model lifecycle:



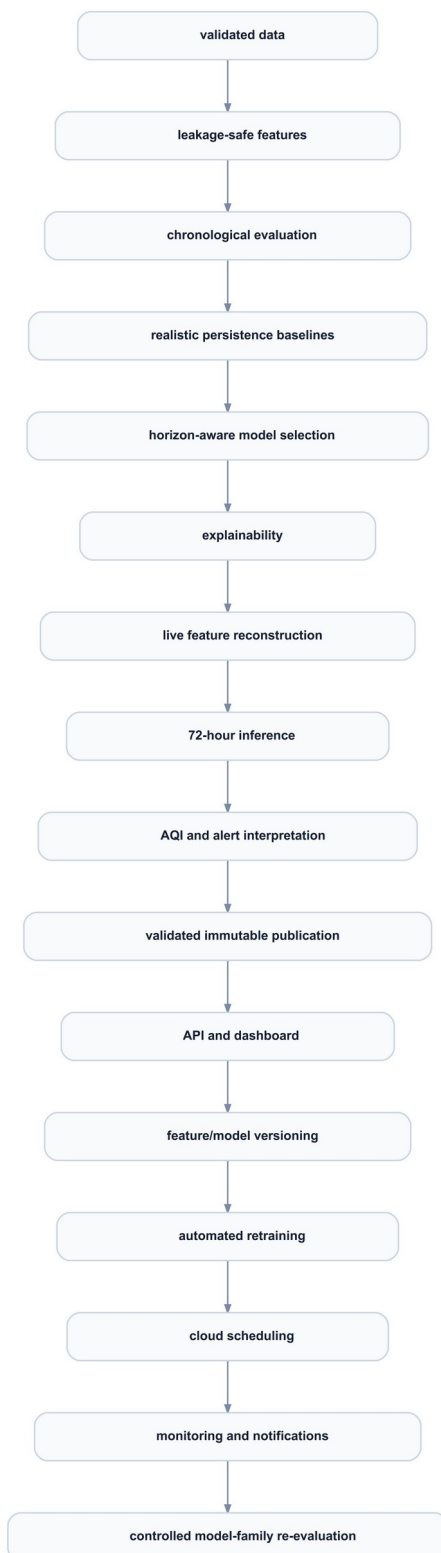
Parallel operational lifecycle:



34. Conclusion

The Pearls AQI Predictor evolved from a historical PM2.5 forecasting exercise into a complete cloud-deployed data product. The most important achievement is not simply that an XGBoost model was

trained. The project demonstrates a full decision and operational chain:



The final production model was challenged rather than accepted unquestioningly.

The strongest HistGradientBoosting challenger improved validation metrics but slightly worsened final historical test RMSE. LightGBM also produced competitive validation RMSE, but the gains were

not sufficient to justify replacing a proven production champion.

The final decision to retain the hybrid persistence/XGBoost model is therefore evidence-based.

The project demonstrates successful work across:

- data collection;
- data validation;
- feature engineering;
- machine learning;
- model evaluation;
- explainability;
- MLOps;
- backend development;
- dashboard development;
- containerization;
- Azure cloud deployment;
- scheduled automation;
- monitoring;
- model lifecycle management;
- production migration;
- operational reliability;
- cost optimization;
- technical documentation.

The completed system is a reproducible, automated, versioned, monitored, and cloud-deployed 72-hour PM2.5-based AQI forecasting platform.